

École Nationale d'Ingénieurs de Brest

Module ReV — Réalité Virtuelle

Conception d'un musée virtuel en Babylon.js

Modélisation, animation et simulation de foule

Juan Marcos De Miguel Funes

Julian Rayes

Encadrant : Éric Maisel

Brest — Juin 2026

Table des matières

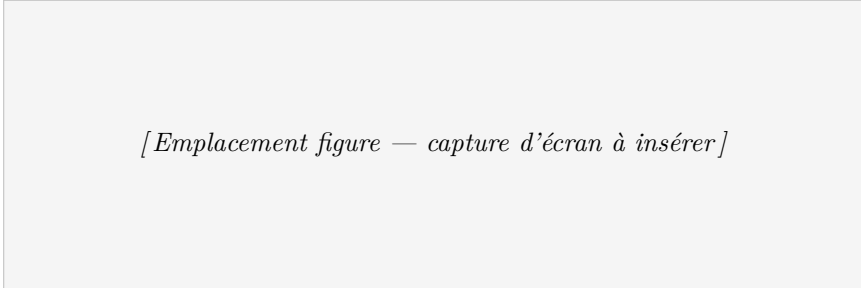
1	Introduction	2
2	Architecture logicielle	2
3	Modélisation et matériaux	3
3.1	Structure, ouvertures et collisions	3
3.2	Escalier, garde-corps et habillage	3
4	Tableaux et sculptures	3
4.1	Tableaux	3
4.2	Statues importées et mobiles animés	4
5	Portes coulissantes automatiques	5
6	Visiteurs autonomes : steering et arbres de comportement	5
6.1	Steering et trajectoire	6
6.2	Arbre de comportement	6
7	Simulation de foule : guide et groupe	7
7.1	Règles de Reynolds	7
7.2	Guide et suiveurs	7
8	Problèmes rencontrés et solutions	7
9	Conclusion	8

1 Introduction

Ce rapport décrit la conception d'une *visite de musée virtuel* réalisée avec le moteur 3D temps réel **Babylon.js**, dans le cadre du module ReV. Le bâtiment occupe une emprise de 30×30 m : un hall vitré de 15×30 m et de 10 m de haut au sud, et au nord trois salles fermées par des portes coulissantes, surmontées d'une mezzanine accessible par un escalier. Les salles exposent des tableaux ; le hall et la mezzanine accueillent des statues et un mobile articulé.

Le projet a été développé en groupe et étalé sur plusieurs séances. Une première base a été posée — squelette de l'application (couches **Visu/Simu**), système entités/composants, premières primitives, bâtiment initial et tableaux — puis progressivement étendue : portes automatiques, sculptures, décor extérieur, garde-corps, et enfin une simulation de visiteurs autonomes et de foule guidée. La collection d'oeuvres est volontairement *éclectique*, mêlant chefs-d'oeuvre classiques (la Joconde, la Création d'Adam, la Nuit étoilée) et images plus personnelles ; elle sert de support aux fonctionnalités techniques qui font le coeur du projet.

L'application utilise les modules ES ; elle se lance en servant le dossier en HTTP (`python3 -m http.server`) puis en ouvrant `index.html`, qui charge Babylon, instancie la classe `World` et démarre la boucle de rendu.



[Emplacement figure — capture d'écran à insérer]

Figure 1: Vue d'ensemble du musée depuis l'extérieur : hall vitré, parvis et ciel.

2 Architecture logicielle

Le projet repose sur une architecture **entités-composants (ECS)** inspirée du cours *Simuler pour animer*. Une `Entity` (`js/entities/entities.js`) est un conteneur — nom, position, `object3d`, liste de composants — doté d'une API chaînable (`createEntity(...).add(...)`). Deux spécialisations gèrent le mouvement : `Kine` (cinématique) et `Newton` (forces $\rightarrow$ accélération $\rightarrow$ vitesse $\rightarrow$ position, par intégration d'Euler).

Le point clé est la **double passe** de `Simu.update(dt)` : chaque image, on *décide* d'abord (tous les composants calculent leurs forces et font évoluer les arbres de comportement), puis on *intègre* (le mouvement résultant est appliqué). Un composant d'IA ne touche donc jamais l'état physique en cours de calcul.

```
1 update(dt) {
2   this.entities.forEach((e) => { if (e.execute) e.execute(dt); }); // 1. decision
3   this.entities.forEach((e) => { if (e.update) e.update(dt); }); // 2.
      integration
4 }
```

Listing 1: Double passe décision / intégration (`js/simu.js`).

Les composants ne se connaissent pas entre eux : ils communiquent par un *blackboard* (`entity.blackboard`). La trajectoire y écrit le point visé, le steering le lit, l'arbre de comporte-

ment y stocke l'état de pause, le guide y publie le tableau présenté. Ce découplage a permis d'empiler de nouveaux comportements sans réécrire l'existant.

3 Modélisation et matériaux

3.1 Structure, ouvertures et collisions

Les cloisons sont produites par `js/prims.js`. Les ouvertures (portes, baies) sont creusées par **opérations CSG** : la fonction `creuser` soustrait au mur un volume-outil (`BABYLON.CSG.subtract`). Le mur intérieur qui sépare le hall des salles est ainsi percé de trois portes, et les baies vitrées reçoivent un cadre et une vitre semi-transparente. La caméra est une `UniversalCamera` en vue première personne, avec capsule de collision, gravité et déplacements QWERTY ; tous les éléments structurels sont collisionnables.

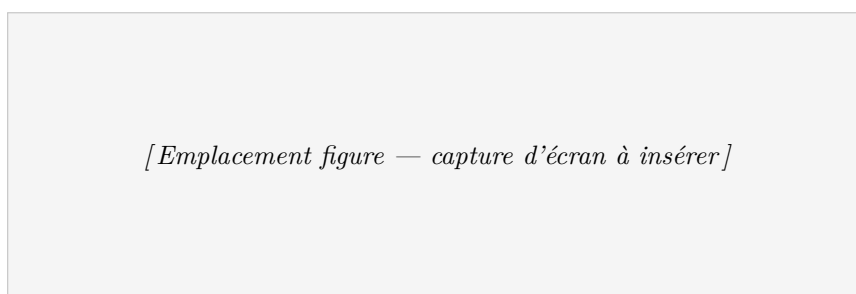


Figure 2: Hall vu de l'intérieur : baies vitrées et mur à trois portes coulissantes.

3.2 Escalier, garde-corps et habillage

L'escalier (20 marches) double chaque marche : une *planche visible* fine et un *bloc de collision invisible* dont la hauteur croît avec l'indice. C'est ce bloc qui est collisionnable, ce qui évite à la caméra de rester bloquée ou de sauter de marche en marche — problème classique des escaliers. Le bord ouvert de la mezzanine est protégé par un **garde-corps en verre** collisionnable, interrompu là où débouche l'escalier pour laisser le passage.

Côté matériaux, le musée combine teintes unies (murs gris à deux variantes, tapis, piédestaux) et textures (sol intérieur, skybox). Le parvis extérieur est une *pelouse procédurale* : une `DynamicTexture` 16×16 remplie au hasard d'une palette de verts et échantillonnée en mode NEAREST (rendu « bloc » façon Minecraft), sans aucune dépendance à un fichier image.

4 Tableaux et sculptures

4.1 Tableaux

Chaque tableau est un *poster* double-face (deux plans dos à dos partageant la même texture). Il est *accroché* en devenant **parent** du mur cible, puis positionné en **coordonnées locales** : un décalage perpendiculaire le décolle du mur et choisit la face exposée, une rotation l'oriente vers l'intérieur. Les positions évitent les baies vitrées. Une quinzaine de tableaux sont répartis dans les salles et sur la mezzanine.

[Emplacement figure — capture d’écran à insérer]

Figure 3: Une salle d’exposition : tableaux accrochés aux murs.

4.2 Statues importées et mobiles animés

Le musée présente trois familles de sculptures.

Statue tournante. La sculpture *La Valse* (.glb) est importée via `ImportMesh` et tourne lentement sur un piédestal. La fonction d’import dispose d’une option `poserAuSol` qui, après mise à l’échelle, calcule la boîte englobante pour poser la base au sol et centrer l’empreinte du modèle sur son axe — sans quoi elle traversait le sol et tournait de travers.

[Emplacement figure — capture d’écran à insérer]

Figure 4: La statue *La Valse*, importée et tournant sur son piédestal.

Maillages statiques. Trois animaux low-poly (chat, cheval, loup) ornent le hall d’entrée, alignés face à la porte et rendus collisionnables (obstacles). Leur orientation est fixée dans le `callback onCharge`, exécuté *après* le centrage automatique — la fixer avant aurait décentré le modèle.

[Emplacement figure — capture d’écran à insérer]

Figure 5: Les animaux low-poly importés dans le hall, faisant face à l’entrée.

Mobile articulé (Calder). Au-dessus de la mezzanine pend un mobile entièrement géométrique, construit *récurivement* : chaque niveau est une tige pivotant sur un axe vertical, portant un disque coloré d’un côté et le niveau inférieur (suspendu par un fil) de l’autre. Le composant `Mobile` fait tourner chaque tige à sa propre vitesse, en sens opposé d’un niveau à l’autre ; le mouvement est donc *articulé* (les parties bougent les unes par rapport aux autres), contrairement à la rotation d’un bloc rigide de *La Valse*.

[Emplacement figure — capture d’écran à insérer]

Figure 6: Le mobile articulé (style Calder) suspendu au-dessus de la mezzanine.

5 Portes coulissantes automatiques

Les quatre portes (entrée principale et trois portes internes) s’ouvrent à l’approche d’un agent et se referment à son éloignement, en suivant la directive du cours (« regarder régulièrement ») : la proximité est évaluée à *chaque pas*, pas par un raycast ponctuel. Chaque battant mémorise sa position fermée et ouverte ; le composant `AutoDoor` mesure la distance de l’agent le plus proche (caméra ou visiteur) et interpole en douceur les battants.

```
1 execute(dt){
2   const open = this.nearestAgentDistance() < this.radius;
3   for (const p of this.panels) {
4     const targetX = open ? p.openX : p.closedX;
5     p.mesh.position.x += (targetX - p.mesh.position.x) * this.speed; // lissage
6   }
7 }
```

Listing 2: Ouverture par proximité (`js/components/autoDoor.js`).

Le dimensionnement des battants a demandé quelques itérations : fermés, les battants laissaient voir l’intérieur par les jointures. La version finale les surdimensionne uniquement vers leurs bords *exposés* (extérieur et haut), en laissant le bord intérieur (là où les deux battants se rejoignent) au centre exact, sans quoi ils se chevauchaient.

[Emplacement figure — capture d’écran à insérer]

Figure 7: Une porte coulissante s’ouvrant à l’approche d’un visiteur.

6 Visiteurs autonomes : steering et arbres de comportement

Le coeur dynamique du musée est une population de visiteurs autonomes : ils marchent dans une salle, franchissent une porte, s’arrêtent devant une oeuvre, puis repartent en boucle. Chaque visiteur combine trois mécanismes indépendants reliés par le blackboard.

6.1 Steering et trajectoire

Seek et **Arrive** (d'après Reynolds, 1987) appliquent une force de pilotage $\vec{F}_s = k(\vec{v}_d - \vec{v}_c)$ vers le point visé ; **Arrive** ralentit dans un rayon pour s'arrêter sans osciller, et c'est lui qu'utilisent les visiteurs. **Trajectory** enchaîne des points de passage (avec bouclage, point de départ et sens réglables, pour désynchroniser plusieurs visiteurs sur le même circuit). Ces points sont calculés à partir des coordonnées réelles des murs et des portes, pour que chaque segment traverse bien une ouverture. Enfin, comme la caméra, chaque visiteur possède un **collider invisible** déplacé par `moveWithCollisions` ; le corps visible (peau, vêtements, cheveux, yeux) en est l'enfant, et Babylon le fait glisser le long des murs.

6.2 Arbre de comportement

Le module `js/bt/` fournit les primitives classiques : **Selector** (« OU »), **Sequence** (« ET »), décorateurs, **Condition** et **Action**, renvoyant chacune un **Status** (SUCCESS, FAILURE, RUNNING).

```
1 class Selector extends Node { // "OU" : premier non-FAILURE
2   tick(entity, dt) {
3     for (const child of this.children) {
4       const s = child.tick(entity, dt);
5       if (s !== Status.FAILURE) return s;
6     }
7     return Status.FAILURE;
8   }
9 }
10 class Sequence extends Node { // "ET" : premier non-SUCCESS
11   tick(entity, dt) {
12     for (const child of this.children) {
13       const s = child.tick(entity, dt);
14       if (s !== Status.SUCCESS) return s;
15     }
16     return Status.SUCCESS;
17   }
18 }
```

Listing 3: Noeuds composites Selector et Sequence (`js/bt/behaviorTree.js`).

L'arbre du visiteur ne gouverne que les *transitions d'état* (marcher ↔ en pause) : une branche « est en pause » décompte un minuteur puis avance au point suivant, une branche « est arrivé » déclenche la pause. Si aucune ne se déclenche, le Selector renvoie **FAILURE** — c'est-à-dire « continuer à marcher », ce que le steering fait déjà seul. Choisir un arbre de comportement plutôt qu'une machine à états le rend *composable* : ajouter un état revient à ajouter une branche.

[Emplacement figure — capture d'écran à insérer]

Figure 8: Plusieurs visiteurs parcourant le musée et s'arrêtant devant les tableaux.

7 Simulation de foule : guide et groupe

La dernière étape étend les visiteurs en une *simulation de foule*, en s'appuyant sur le cours consacré aux comportements collectifs, et toujours par-dessus l'infrastructure ECS existante.

7.1 Règles de Reynolds

Trois composants implémentent le flocking, chacun lisant ses voisins dans un tableau partagé : **Separation** (répulsion $\propto 1/d^2$), **Cohesion** (attraction vers le barycentre du groupe) et **Alignment** (alignement sur la vitesse moyenne).

```
1 execute(dt){
2   const force = new BABYLON.Vector3(0,0,0);
3   const p = this.entity.position;
4   for (const other of this.group){
5     if (other === this.entity || !other.position) continue;
6     const away = p.subtract(other.position); // pointe a l'oppose du voisin
7     const dist = away.length();
8     if (dist > 0.0001 && dist < this.radius)
9       force.addInPlace(away.scale(1.0 / (dist * dist)));
10  }
11  this.entity.applyForce(force.scale(this.k));
12 }
```

Listing 4: Force de séparation, répulsion en $1/d^2$ (js/components/separation.js).

7.2 Guide et suiveurs

Un **guide** (entité plus grande, dorée) suit sa propre trajectoire et s'arrête devant certains tableaux ; à l'arrêt, son arbre de comportement publie le tableau présenté dans le blackboard, les points sans arrêt étant simplement traversés pour mener le groupe d'une salle à l'autre. Quatre **suiveurs** visent, via **FollowGuide**, un point situé *derrière* le guide (et non le guide lui-même, ce qui les ferait s'empiler), avec un freinage type *Arrive*. Quand le guide présente une oeuvre, le composant **Gaze** tourne chaque suiveur vers le même tableau ; le guide reparti, l'orientation revient à la direction de marche. Les murs et le parcours du guide confinant déjà le groupe, aucun mécanisme de rebond n'a été nécessaire. La scène compte au final un guide, quatre suiveurs et six visiteurs libres (trois en bas, trois sur la mezzanine).

[Emplacement figure — capture d'écran à insérer]

Figure 9: Le guide (doré) présentant un tableau ; le groupe se rassemble et tourne le regard vers l'oeuvre.

8 Problèmes rencontrés et solutions

Visiteurs qui flottaient. Avec un groupe nombreux, les personnages se tassaient, se grimpèrent dessus et finissaient en l'air, et bloquaient les portes (2 m). En cause : leurs colliders entraient

en collision *entre eux*, et les forces de cohésion les poussaient vers le haut sans gravité pour les rabaisser. La solution a été de *renoncer aux collisions entre visiteurs* (groupes de collision : ils ignorent les autres personnes mais heurtent murs et portes) et de confier l'espacement à la force de séparation de Reynolds, avec deux garde-fous : un mode **planar** (vitesse et position recollées au sol) et un plafond de vitesse.

Le guide traversait les visiteurs. Conséquence directe : sans collision mutuelle, le guide passait à travers le groupe. Plutôt que de rétablir les collisions, une *répulsion d'espace personnel* (composant **AvoidGuide**, et logique équivalente dans **FollowGuide**) écarte les visiteurs du chemin du guide dans un petit rayon. Le déclenchement utilise la distance 3D (un guide à un autre étage n'affecte personne) et la poussée reste horizontale.

Le cheval, ancien centre des trajectoires. Devenu obstacle solide, le cheval occupait le point central vers lequel revenaient toutes les trajectoires : les visiteurs s'y agglutinaient et le guide restait coincé. Le point de ralliement a été déplacé derrière le cheval, hors de son empreinte de collision, comme les positions de départ du groupe.

D'autres ajustements plus ponctuels (statue traversant le sol, disques du mobile qui semblaient flotter faute de fils de liaison, dimensionnement des portes) ont été diagnostiqués à partir de captures d'écran prises pendant les tests.

9 Conclusion

Le projet aboutit à un musée virtuel cohérent sur les plans statique et dynamique : un bâtiment complet (hall, salles, mezzanine, escalier, baies, garde-corps), habillé de matériaux variés, peuplé de tableaux, de statues importées et d'un mobile articulé, doté de portes réactives, et animé par des visiteurs autonomes jusqu'à une foule guidée. L'apport principal est *architectural* : une base entités-composants soignée a permis d'empiler des comportements (steering, arbres de comportement, flocking, évitement) par simple composition, et de corriger les problèmes au fur et à mesure sans jamais réécrire l'existant.